

Skeletal Mesh 퀴즈 정리

WEEK10 발제문 기반 - 문제와 답안 통합본

기존 선별 문항의 빠진 번호는 무시하고, 1 번부터 새로 번호를 매겨 정리했다.

1. Static Mesh 와 Skeletal Mesh 의 차이

문제 Static Mesh 와 Skeletal Mesh 의 가장 큰 차이는 무엇인가?

답

Static Mesh 는 기본적으로 정점들이 고정된 메시다. 메시 자체가 Bone 이나 애니메이션 구조에 의해 변형되지 않는다.

Skeletal Mesh 는 Bone 계층 구조와 Vertex Weight 정보를 가진다. 각 정점은 하나 이상의 Bone 영향을 받아 움직일 수 있고, 이 구조 덕분에 캐릭터의 팔, 다리, 허리처럼 관절을 기준으로 자연스럽게 변형되는 모델을 표현할 수 있다.

2. Root Bone 이 필요한 이유

문제 Root Bone 이 필요한 이유는 무엇인가?

답

Root Bone 은 Skeleton 전체의 기준점 역할을 한다. 모든 Bone 은 부모-자식 관계를 이루며 계층 구조를 만들고, 그 계층의 최상위에 있는 Bone 이 Root Bone 이다.

Root Bone 이 있어야 캐릭터 전체의 이동, 회전, 스케일을 하나의 기준에서 관리할 수 있다. 예를 들어 캐릭터가 앞으로 이동하거나 몸 전체가 회전할 때, 개별 팔이나 다리 Bone 을 따로 움직이는 것이 아니라 Root Bone 또는 Root 계층을 기준으로 전체 Skeleton 이 함께 움직인다.

3. Reference Pose, Bind Pose, T-Pose

문제 Reference Pose, Bind Pose, T-Pose 는 각각 어떤 의미인가? 이 셋은 완전히 같은 개념인가?

답

Reference Pose 는 Skeletal Mesh 가 기준으로 삼는 기본 자세다. 엔진에서 메시를 처음 불러왔을 때의 기준 자세라고 볼 수 있다.

Bind Pose 는 Mesh 가 Skeleton 에 묶였을 때의 자세다. 이 자세를 기준으로 각 Vertex 가 어떤 Bone 의 영향을 얼마나 받는지 정의된다.

T-Pose 는 양팔을 좌우로 벌린 형태의 자세를 말한다. 많은 캐릭터 모델이 T-Pose 를 Reference Pose 나 Bind Pose 로 사용하지만, 모든 Reference Pose 나 Bind Pose 가 반드시 T-Pose 인 것은 아니다. T-Pose 는 자주 쓰이는 형태이고, Reference Pose 와 Bind Pose 는 더 일반적인 개념이다.

4. Vertex 당 Bone Influence 수를 제한하는 이유

문제 한 Vertex 가 영향을 받을 수 있는 Bone 수를 보통 4 개로 제한하는 이유는 무엇인가?

답

주된 이유는 성능과 메모리다. 한 Vertex 가 너무 많은 Bone 의 영향을 받으면 Skinning 계산량이 늘어난다. 또한 각 Vertex 마다 Bone Index 와 Weight 를 더 많이 저장해야 하므로 Vertex Buffer 크기도 커진다.

실시간 렌더링에서는 보통 한 Vertex 당 최대 4 개의 Bone Influence 만 사용한다. 4 개 정도면 대부분의 캐릭터 변형을 충분히 표현하면서도 계산 비용을 적절히 유지할 수 있다.

5. FBX 의 Control Point 와 Vertex

문제 FBX 에서 Control Point 와 Vertex 의 차이는 무엇인가? 이 둘을 헷갈리면 어떤 문제가 생길 수 있는가?

답

Control Point 는 FBX 에서 원본 위치 데이터에 가까운 개념이다. 쉽게 말하면 모델을 구성하는 기본 좌표점이다.

렌더링에 사용하는 Vertex 는 단순히 위치만 가진 점이 아니다. Normal, UV, Tangent, Material Index 같은 렌더링 속성을 함께 가진 단위다. 같은 위치를 공유하더라도 면마다 Normal 이나 UV 가 다르면 렌더링 Vertex 는 여러 개로 나뉘어야 한다.

이 둘을 같은 것으로 처리하면 UV 가 깨지거나 Normal 이 이상해질 수 있고, Bone Weight 가 잘못 연결될 수도 있다. 특히 FBX 의 Skinning 정보는 Control Point Index 를 기준으로 들어오는 경우가 많으므로, 최종 렌더링 Vertex 를 만들 때 Bone Index 와 Weight 를 올바르게 복사해야 한다.

6. Control Point 가 여러 Vertex 로 분리되는 이유

문제 하나의 Control Point 가 여러 개의 렌더링 Vertex 로 분리될 수 있는 이유는 무엇인가?

답

같은 위치를 가진 점이라도 면마다 다른 속성을 가질 수 있기 때문이다.

예를 들어 큐브의 한 모서리는 위치상으로는 하나의 점이지만, 각 면에서 바라보는 Normal 방향이 다르다. UV 도 면마다 다를 수 있고, Material 도 달라질 수 있다.

따라서 FBX 의 Control Point 하나가 엔진의 렌더링 Vertex 하나와 항상 1:1 로 대응된다고 보면 안 된다. 최종 Vertex 는 위치뿐만 아니라 Normal, UV, Tangent, Material 같은 렌더링 속성까지 포함한 단위로 만들어져야 한다.

7. Material, Normal, UV 와 Bone 정보의 구분

문제 FBX 에서 Material, Normal, UV 정보는 Bone 정보와 같은 계층에 있는가? 아니면 Mesh Attribute 쪽에 가까운가?

답

Material, Normal, UV 는 Bone 정보라기보다는 Mesh Attribute 쪽에 가깝다.

Bone 과 Weight 정보는 Skinning 을 위한 데이터이며, 보통 Skin, Cluster 구조를 통해 Mesh 에 연결된다. 반면 Normal, UV, Material 은 Mesh 의 표면을 어떻게 렌더링할지 결정하는 데이터다.

따라서 FBX 를 파싱할 때는 Geometry 정보와 Skinning 정보를 분리해서 생각해야 한다. Mesh 는 정점, 인덱스, Normal, UV, Material 정보를 가지고, Skin 또는 Cluster 는 어떤 Bone 이 어떤 Control Point 에 얼마나 영향을 주는지를 가진다고 보면 된다.

8. Linear Blend Skinning의 기본 원리

문제 LBS, 즉 Linear Blend Skinning의 기본 원리를 설명하라.

답

Linear Blend Skinning은 한 Vertex의 최종 위치를 여러 Bone의 변형 결과를 가중합해서 구하는 방식이다.

각 Bone은 Vertex를 자기 Transform에 따라 변환한다. 그 결과에 해당 Bone의 Weight를 곱하고, 모든 Bone의 결과를 더하면 최종 Vertex 위치가 나온다.

예를 들어 어떤 Vertex가 UpperArm Bone에 0.7, LowerArm Bone에 0.3의 영향을 받는다면, UpperArm 기준으로 변형된 위치를 70%, LowerArm 기준으로 변형된 위치를 30% 반영해서 최종 위치를 만든다.

9. Skinning Matrix 식의 의미

문제 다음 식에서 각 항이 의미하는 바를 설명하라.

```
skinningMatrix = BoneTransform * InverseBindPose;
```

답

BoneTransform은 현재 Pose에서의 Bone Transform이다. Skinning에 사용할 때는 보통 Local Transform이 아니라 부모 Bone들의 Transform까지 누적된 Component Space 또는 Model Space Transform이 필요하다.

InverseBindPose는 Bind Pose 상태에서의 Bone Transform을 역행렬로 만든 것이다. 이 행렬은 Vertex를 Bind Pose에서의 Bone 기준 공간으로 되돌리는 역할을 한다.

BoneTransform * InverseBindPose는 Bind Pose에 있던 Vertex를 현재 Bone Pose에 맞게 이동시키는 행렬이다. 즉, 이 행렬이 실제 Skinning에 사용되는 Bone 별 최종 변형 행렬이다.

단, 실제 곱셈 순서는 엔진의 행렬 규칙에 따라 달라질 수 있다. Row Vector를 쓰는지 Column Vector를 쓰는지, Row-major인지 Column-major인지에 따라 코드의 곱셈 순서는 다르게 보일 수 있다.

10. Inverse Bind Pose Matrix가 필요한 이유

문제 Inverse Bind Pose Matrix가 필요한 이유는 무엇인가? 현재 Bone Transform만 Vertex에 적용하면 왜 안 되는가?

답

Vertex는 이미 Bind Pose 상태의 Mesh 공간에 놓여 있다. 즉, 모델링된 정점 위치 자체가 기준 Skeleton 자세에 맞춰져 있다.

여기에 현재 Bone Transform만 바로 적용하면, Bind Pose에 들어 있던 기준 변형을 제거하지 않은 상태에서 다시 Bone Transform을 적용하는 셈이 된다. 그 결과 Vertex가 잘못된 위치로 이동하거나 관절 주변이 심하게 틀어질 수 있다.

Inverse Bind Pose는 먼저 Vertex를 Bind Pose에서의 Bone 기준 위치로 되돌린다. 그 다음 현재 Bone Transform을 적용해야 현재 Pose에 맞는 올바른 Vertex 위치가 나온다.

11. Reference Pose 에서 원래 Mesh 가 유지되는 조건

문제 Reference Pose 상태에서 Skinning 결과가 원래 Mesh 와 같게 나오려면 어떤 조건이 성립해야 하는가?

답

현재 Bone Transform 이 Bind Pose 의 Bone Transform 과 같아야 한다.

이 경우 Skinning Matrix 는 $\text{BindPoseBoneTransform} * \text{InverseBindPose}$ 형태가 된다. Bind Pose Transform 에 그 역행렬을 곱하면 결과는 거의 Identity Matrix 가 된다. 그러면 Vertex 는 변형되지 않고 원래 Reference Pose 의 위치에 그대로 남는다.

Reference Pose 에서 Skinning 을 수행했는데 Mesh 가 터지거나 움직인다면, Inverse Bind Pose 계산, Bone Transform 누적, 행렬 곱 순서, 좌표계 변환 중 하나가 잘못되었을 가능성이 높다.

12. Bone Transform 의 Space 와 Skinning 에 필요한 Transform

문제 Bone Transform 은 보통 Local Space 기준인가, Component Space 기준인가? Skinning 에 최종적으로 필요한 것은 어느 쪽인가?

답

Skeleton 계층에 저장된 각 Bone 의 Transform 은 보통 Local Space 기준이다. 즉, 각 Bone 은 자기 부모 Bone 을 기준으로 한 위치, 회전, 스케일 값을 가진다.

하지만 Skinning 에 최종적으로 필요한 것은 부모 Transform 이 모두 누적된 Component Space 또는 Model Space 기준의 Bone Transform 이다.

예를 들어 손 Bone 의 최종 위치는 손 자체의 Local Transform 만으로 결정되지 않는다. 팔뚝, 위팔, 어깨, 척추, 골반 같은 부모 Bone 들의 Transform 이 모두 누적되어야 실제 손의 위치가 나온다.

그래서 엔진에서는 보통 먼저 Local Pose 를 만들고, Bone Hierarchy 를 따라 Component Space Pose 로 변환한 다음, 이 값을 Inverse Bind Pose 와 결합해 Skinning Matrix 를 만든다.

13. Normal 과 Tangent 에도 Skinning 이 필요한 이유

문제 Position 뿐 아니라 Normal 과 Tangent 에도 Skinning 이 필요한 이유는 무엇인가?

답

Position 만 Skinning 하면 Mesh 의 모양은 움직이지만 표면 방향 정보는 원래 자세에 남게 된다.

Normal 은 조명 계산에 사용되고, Tangent 는 Normal Mapping 에 사용된다. 따라서 Vertex Position 이 Bone 에 의해 변형되었다면 Normal 과 Tangent 도 같은 Bone Weight 를 기준으로 함께 변형되어야 한다.

그렇지 않으면 캐릭터의 팔이나 다리는 움직였는데 조명은 예전 방향을 기준으로 계산되어 어색하게 보인다. 특히 Normal Mapping 을 사용하는 모델에서는 표면의 굴곡 표현이 틀어질 수 있다.

14. CPU Skinning의 장점과 단점

문제 CPU Skinning의 장점과 단점을 각각 설명하라.

답

CPU Skinning의 장점은 디버깅이 쉽다는 점이다. Vertex가 어떤 Bone Index와 Weight를 가지고 있는지, Skinning Matrix를 적용한 결과가 어떤 위치로 나오는지 CPU 코드에서 직접 확인할 수 있다.

또한 GPU Shader 경로를 만들기 전에도 Skinning 결과를 확인할 수 있으므로, 교육용 엔진이나 초기 구현 단계에서는 접근하기 쉽다.

단점은 성능 비용이 크다는 점이다. 매 프레임 CPU에서 모든 Vertex를 변형해야 하고, 변형된 결과를 다시 GPU Vertex Buffer에 업로드해야 할 수 있다. Vertex 수가 많거나 캐릭터 수가 많아지면 CPU 연산 비용과 버퍼 업데이트 비용이 빠르게 증가한다.

15. USkinnedMeshComponent와 USkeletalMeshComponent를 나누는 이유

문제 USkinnedMeshComponent와 USkeletalMeshComponent를 나누는 이유는 무엇인가?

```
class USkinnedMeshComponent : public UMeshComponent
{
    USkeletalMesh* SkeletalMesh;
};

class USkeletalMeshComponent : public USkinnedMeshComponent
{
};
```

답

USkinnedMeshComponent는 Skinning이 가능한 Mesh Component의 공통 기반 클래스로 볼 수 있다. Bone이나 Weight에 의해 Vertex가 변형될 수 있는 Mesh라면 공통적으로 가져야 하는 기능을 여기에 둘 수 있다.

USkeletalMeshComponent는 그중에서도 Skeleton과 Bone Hierarchy를 사용하는 구체적인 Component다. 실제 캐릭터처럼 Skeletal Mesh 리소스를 가지고, Bone Transform을 갱신하고, Skinning Matrix를 계산하고, 애니메이션 Pose를 적용하는 역할을 맡는다.

이렇게 나누면 구조가 확장하기 쉬워진다. 나중에 Morph Target, Cloth, 다른 종류의 Skinned Mesh 기능이 추가되더라도 공통 기능은 USkinnedMeshComponent에 두고, Skeleton 기반 기능은 USkeletalMeshComponent에 둘 수 있다.